20 · Production Projects and Labs Blueprint

> **Goal:** turn the reading in chapters 1-19 into five portfolio-grade projects with milestones, acceptance tests, failure drills and an evaluation rubric, so every claim in an interview is backed by something you built and broke on purpose.

Table of Contents

- 20.1 Project 1: Reproducible Training Pipeline with Validation and Tracking
- 20.2 Project 2: Feature Store and Online Serving with Parity Tests
- 20.3 Project 3: CI/CD/CT with Gated Promotion and Progressive Delivery
- 20.4 Project 4: Monitoring, Drift and the Closed Retraining Loop
- 20.5 Capstone: End-to-End Streaming Fraud Platform and Review Rubric

Each project uses the same shape: goal, architecture, milestones with acceptance tests, failure drills, stretch goals and the interview story it produces. The code in each section is a stand-in that runs with the standard library and numpy only, so you can verify your own harness before wiring in DVC, MLflow, Feast, Redis or a cluster. Use synthetic data throughout. The 30-day schedule in `COURSE-30-DAY.md` already contains small labs; these projects are the larger, portfolio-level versions, and 21.2 shows where to fit them into a study plan.

20.1 Project 1: Reproducible Training Pipeline with Validation and Tracking

Conceptual Deep Dive

Goal. A repository a stranger can clone and reproduce with one command. Interviewers look for evidence that you treat code, data and parameters as versioned inputs: a pinned environment, a data contract that fails loudly, tracked metrics with lineage, and a README that states how to reproduce a past result.

Architecture. A DVC pipeline with stages validate, train, evaluate and register; `params.yaml` as the single source of hyperparameters; MLflow tracking for runs and metrics; a Dockerfile for the environment; data contract tests that run as the first stage (Module 01 section 5, Module 02 section 1 and Module 03 section 4 (legacy)). Large data lives in a remote, never in Git.

Milestones. M1: pipeline runs end to end on a synthetic dataset. M2: validation stage with schema, range and null-rate checks. M3: every run logs params, metric, data hash, git SHA and seed. M4: reproduce an old run from a git tag alone. M5: break the data on purpose, watch the pipeline fail at the validate stage, and rerun only the changed stages.

Acceptance tests. Two runs from the same tag produce metrics equal within a stated tolerance; a corrupted input stops the pipeline before training; changing one parameter reruns only the dependent stages; a dirty git tree blocks registration.

Definition of done. README with Quickstart, Reproduce, Architecture and Failure drills sections; a one-command reproduction; a table of drills and observed behaviour.

Failure drills. An upstream unit change (dollars to cents), a dirty tree, and a missing seed. For each, record which control caught it and how long detection took.

Stretch. Remote cache shared by two machines, a scheduled rerun that compares to the last tag, a model card generated from the run. The story it gives you answers questions from 1.15, 2.17 and 3.16.

Edge cases: data too large for a laptop (sample with a fixed seed and document the sampling) and nondeterminism making the reproduction test flaky (fix seeds, then set a tolerance; investigate any gap beyond it).

Cross-Connection Scenarios

- The data tests here are the seed of 13.2; the tolerance logic reappears in 13.3.
- Lineage fields feed registry promotion (4.16) and audit evidence (10.16).
- This pipeline becomes stage one of the capstone in 20.5.
- Orchestration choices and backfill safety: 5.15 and 5.16.

Production Anti-Patterns

- No acceptance tests: "it reproduces on my machine" is a claim, not a result, and nothing catches drift in the pipeline itself.
- Committing data to Git: history bloats, access control is lost and nobody can say which data trained which model.
- A project with no README on how to reproduce: the work cannot be verified, so it earns no credit in a review.

Minimal Working Code

An acceptance-test runner that checks files, README sections and run equality.

```
import json, tempfile
from pathlib import Path

REQUIRED = ["dvc.yaml", "params.yaml", "README.md"]
SECTIONS = ["Reproduce", "Architecture", "Failure drills"]

def metrics_match(a, b, tol=1e-9):
    ma, mb = json.loads(Path(a).read_text()), json.loads(Path(b).read_text())
    nums = all(abs(v - mb[k]) <= tol for k, v in ma.items() if isinstance(v, float))
    return nums and ma.get("data_hash") == mb.get("data_hash")

def acceptance(root, run_a, run_b, tol=1e-9):
    root = Path(root)
    readme = (root / "README.md").read_text() if (root / "README.md").exists() else &
quot;"
    checks = [(f"file {f}", (root / f).exists()) for f in REQUIRED]
    checks += [(f"README: {s}", s.lower() in readme.lower()) for s in SECTIONS]
    checks.append(("two runs match", metrics_match(run_a, run_b, tol)))
    for name, ok in checks:
        print(("[x] " if ok else "[ ] ") + name)
    return all(ok for _, ok in checks)

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as d:
        root = Path(d)
        for f in REQUIRED[:2]:
            (root / f).write_text("x")
        (root / "README.md").write_text("## Reproduce\n## Architecture\n")
        m = json.dumps({"accuracy": 0.91, "data_hash": "abc"})
        (root / "a.json").write_text(m)
        (root / "b.json").write_text(m)
        assert acceptance(root, root / "a.json", root / "b.json") is False
```

A stand-in pipeline that validates, trains a tiny model and writes metrics with lineage.

```
import hashlib, json, tempfile
import numpy as np
from pathlib import Path

def generate(seed, n=400):
    rng = np.random.default_rng(seed)
    X = rng.normal(size=(n, 3))
    y = (X @ np.array([1.5, -2.0, 0.5]) + rng.normal(scale=0.5, size=n) > 0) * 1.0
    return X, y

def validate(X, y):
    if not np.isfinite(X).all():
        raise ValueError("non-finite feature")
    if abs(X.mean()) > 1:
        raise ValueError("feature mean outside contract")

def run(out_dir, seed=7, corrupt=False):
    X, y = generate(seed)
    if corrupt:
        X[0, 0] = np.nan
    validate(X, y)
    w = np.zeros(3)
    for _ in range(200):
        p = 1 / (1 + np.exp(-X @ w))
        w -= 0.5 * X.T @ (p - y) / len(y)
    acc = float((((X @ w) > 0) == y).mean())
    lineage = hashlib.sha256(X.tobytes() + y.tobytes()).hexdigest()[:12]
    path = Path(out_dir) / "metrics.json"
    path.write_text(json.dumps({"accuracy": acc, "seed": seed, "data_hash":
 lineage}))
    return path

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as d:
        a, b = Path(d, "a"), Path(d, "b")
        a.mkdir(), b.mkdir()
        assert metrics_match(run(a), run(b))
        try:
            run(a, corrupt=True)
        except ValueError as e:
            print("blocked:", e)
        print(json.loads(run(a).read_text()))
```

Interview Angle

Q: Walk me through your reproducible pipeline project; what broke and how did you detect it? **A:** State the stages, then a real drill: I changed a unit upstream and the validate stage failed on a range check before training started. Trade-off: strict contracts add false alarms and maintenance. Measurable check: time from corrupt input to a failed pipeline (minutes), and a reproduction test that matches metrics within a tolerance across two clean runs. Close with the control I added afterwards, not just the fix.

Q: How would you scale it for a team of 10? **A:** Move state out of laptops: a shared remote for data and cache, CI that runs the reproduction test on every merge, a registry for outputs and a template repository so projects start from the same layout. Trade-off: standardisation slows unusual projects. Check: new-project setup time, share of runs with complete lineage, and the percentage of results a second engineer can reproduce unaided.

20.2 Project 2: Feature Store and Online Serving with Parity Tests

Conceptual Deep Dive

Goal. Batch features, an online lookup and a scoring service, with a training/serving parity test and a freshness SLO. The portfolio value is demonstrating that you understand the central failure of tabular ML in production: features computed one way for training and another way for serving.

Architecture. Offline tables (parquet or a warehouse), a Feast-style registry of entities and feature views (Module 02 sections 2-3 (legacy)), a Redis online store, and a FastAPI service with fallback tiers (Module 11 section 7 (legacy)). Every request logs the features it served, so the same rows can be recomputed offline for comparison.

Milestones. M1: define entities and views. M2: build a point-in-time (PIT) correct training set. M3: materialise to the online store. M4: serve with logging of served features. M5: run a parity job comparing logged values with offline recomputation, then inject staleness and watch the freshness alert.

Acceptance tests. A PIT join test with planted leakage must detect it; the parity mismatch rate stays under a stated tolerance; p99 latency stays under the budget in a load test at 1.5x expected peak; a stale feature set triggers the freshness alert within its window.

Failure drills. Redis down (the service must degrade to defaults or a simple model and report the fallback rate), TTL expiry on idle entities, and late-arriving data that changes what the offline table says about the past.

Stretch. Add one streaming feature (see 14.2 and 14.3) and shadow scoring of a second model (Module 07 section 4 (legacy)). Interview stories map to 2.15, 6.17 and 11.17.

Edge cases: local Redis memory limits (use a smaller entity count and extrapolate with the sizing method in 12.16) and Feast API differences across versions (check current docs; the stand-in below isolates the logic from any particular API).

Cross-Connection Scenarios

- Design questions on ownership and backfills: 2.15; leakage taxonomy: 2.17.
- The fallback counters connect to degradation design in 6.17 and SLO alerting in 8.17.
- Streaming features in this project are specified in 14.2; parity for them in 14.3.
- The capstone in 20.5 reuses this store as its feature layer.

Production Anti-Patterns

- Skipping the parity test: the store is "correct" by construction in the design doc and wrong in production for months.
- Hard-coding defaults silently: missing features become plausible numbers with no counter, so quality decays without any alert.
- No freshness monitoring: a stopped materialisation job looks identical to a healthy system until accuracy drops.

Minimal Working Code

A dict-and-list stand-in store with PIT join and a parity checker.

```
from bisect import bisect_right

class MiniStore:
    def __init__(self):
        self.log = {}          # entity -> sorted [(ts, value)]
        self.online = {}       # entity -> latest value

    def write(self, entity, ts, value):
        self.log.setdefault(entity, []).append((ts, value))
        self.log[entity].sort()
        self.online[entity] = self.log[entity][-1][1]

    def pit(self, entity, label_ts):
        rows = self.log.get(entity, [])
        i = bisect_right([t for t, _ in rows], label_ts)
        return rows[i - 1][1] if i else None

    def leaky(self, entity, label_ts):
        rows = self.log.get(entity, [])
        return rows[-1][1] if rows else None

def parity(store, served_log, tol=1e-9):
    bad = 0
    for entity, ts, served in served_log:
        want = store.pit(entity, ts)
        bad += want is None or abs(want - served) > tol
```

```
    return bad / len(served_log)

if __name__ == "__main__":
    s = MiniStore()
    for ts, v in [(1, 10.0), (5, 20.0), (9, 99.0)]:
        s.write("u1", ts, v)
    s.write("u2", 2, 3.0)
    assert s.pit("u1", 6) == 20.0 and s.leaky("u1", 6) == 99.0   # planted leak caugh
t
    print("mismatch rate:", parity(s, [("u1", 6, 20.0), ("u2", 4, 0.0)]))
```

Fallback-tier scoring with a counter and a test per tier.

```
from collections import Counter

COUNTS = Counter()
DEFAULTS = {"amount": 50.0, "velocity": 1.0}

def model(f):
    return min(1.0, f["amount"] / 1000 + 0.1 * f["velocity"])

def score(features, deny_on_rules=False):
    present = {k: v for k, v in features.items() if v is not None}
    if len(present) == len(DEFAULTS):
        COUNTS["full"] += 1
        return model(present)
    if "amount" in present:
        COUNTS["defaults"] += 1
        return model({**DEFAULTS, **present})
    COUNTS["rules"] += 1
    return 1.0 if deny_on_rules else 0.0

if __name__ == "__main__":
    assert score({"amount": 500.0, "velocity": 2.0}) == 0.7
    assert score({"amount": 500.0, "velocity": None}) == 0.6
    assert score({"amount": None, "velocity": None}, deny_on_rules=True) == 1.0
    print(dict(COUNTS))   # alert when non-"full" share exceeds a threshold
```

Interview Angle

Q: How do you prove training/serving parity in your project? **A:** The service logs every served feature vector; a job recomputes the same rows offline with a PIT join and compares values, reporting the mismatch rate per feature. Trade-off: logging costs storage and a sampling rate may miss rare paths. Measurable check: mismatch rate under a tolerance (for example 0.1%) on a daily sample, with a planted-leak test proving the checker can fail.

Q: What happens in your design when the online store is down? **A:** The service falls through tiers: full features, defaults for missing fields, a simple model, then rules. Each tier increments a counter and the response carries a fallback flag. Trade-off: availability against accuracy and fraud exposure. Check: a drill that kills Redis during a load test, with the fallback rate alert firing and p99 still under budget.

20.3 Project 3: CI/CD/CT with Gated Promotion and Progressive Delivery

Conceptual Deep Dive

Goal. A path from pull request to production in which a bad model is stopped and a degraded release is rolled back automatically. The path is: PR, tests, train, gates, registry, image, staged rollout, analysis, promote or rollback. This project proves you understand that delivery safety is mostly about gates that fail in practice.

Architecture. GitHub Actions (or an equivalent CI system) for tests and the training job (Module 05 section 3 (legacy)); registry aliases for candidate and champion (Module 04 section 2); a promotion script that fails closed; a GitOps repository holding desired

deployment state; and a canary rollout with metric analysis in the style of Argo Rollouts (Module 07 section 6) on kind or minikube. The analysis query must isolate canary pods from stable pods; a selector that matches both lets a healthy stable fleet mask a broken canary.

Milestones. M1: behavioural tests in CI (13.4). M2: a gate script comparing a candidate to the champion on quality, latency and a protected slice. M3: environment protection with human approval for production. M4: canary on a local cluster. M5: force a bad model through every layer.

Acceptance tests. A deliberately bad model is blocked at the gates; a deliberately slow model passes the gates but is rolled back by canary analysis; every release has a traceable bundle (commit, data hash, model version, image digest).

Failure drills. Flaky tests (quarantine with an owner and a deadline), a registry outage (the pipeline must stop, not skip the check) and two concurrent promotions (serialise or use compare-and-set on the alias).

Stretch. A continuous-training trigger with debounce (Module 05 section 9 (legacy), retrain storm). Interview stories map to 4.15, 5.17 and 7.17.

Edge cases: local cluster resource limits (reduce replicas, keep the logic) and rollout or mesh CRD versions varying (check current docs).

Cross-Connection Scenarios

- Tier-based approval rules for the gate policy: 4.15; GitOps and supply chain: 5.17.
- Canary statistics and sequential decisions: 7.17; bandit-style allocation: 7.18.
- Pipeline tests that guard the gate itself: 13.6.
- The same pipeline carries the retrain candidate in the loop of 20.4.

Production Anti-Patterns

- Auto-promote with no champion comparison: any model that clears an absolute bar replaces a better one.
- Gates that never fail in practice: if no candidate has ever been blocked, the thresholds are decorative.
- No rollback test: the first real rollback happens during an incident, with an untested procedure.

Minimal Working Code

A fail-closed gate runner that treats missing metrics as failures.

```
import json

def gate(metrics_json, policy_json):
    m, p = json.loads(metrics_json), json.loads(policy_json)
    reasons = []
    for name, floor in p.get("min", {}).items():
        v = m.get(name)
        if v is None or v < floor:
            reasons.append(f"{name}={v} below {floor}")
    for name, cap in p.get("max", {}).items():
        v = m.get(name)
        if v is None or v > cap:
            reasons.append(f"{name}={v} above {cap}")
    return (1 if reasons else 0), reasons

POLICY = json.dumps({"min": {"auc": 0.90, "slice_recall": 0.80}, "max&
quot;: {"p95_ms": 80}})

if __name__ == "__main__":
    ok = gate(json.dumps({"auc": 0.92, "slice_recall": 0.85, "p95_ms": 60})
, POLICY)
    bad = gate(json.dumps({"auc": 0.95, "p95_ms": 60}), POLICY)   # slice metric miss
ing
    print(ok, bad)
    assert ok == (0, []) and bad[0] == 1
```

A canary analysis simulator that returns promote, hold or abort.

```
import random

def p95(xs):
    xs = sorted(xs)
    return xs[int(0.95 * (len(xs) - 1))]

def decide(stable, canary, min_n=200, err_margin=0.01, lat_ratio=1.2):
    if len(canary["lat"]) < min_n:
        return "hold"
    if canary["err"] > stable["err"] + err_margin:
        return "abort"
    if p95(canary["lat"]) > lat_ratio * p95(stable["lat"]):
        return "abort"
    return "promote"

def sample(seed, n, err, mean_ms):
    r = random.Random(seed)
    return {"err": err, "lat": [r.gauss(mean_ms, mean_ms * 0.2) for _ in range(n)]}

if __name__ == "__main__":
    stable = sample(1, 1000, 0.010, 40)
    assert decide(stable, sample(2, 50, 0.010, 40)) == "hold"
    assert decide(stable, sample(3, 500, 0.010, 41)) == "promote"
    assert decide(stable, sample(4, 500, 0.010, 70)) == "abort"
    print("canary decisions ok")
```

Interview Angle

Q: Show me how a bad model is stopped in your pipeline. **A:** Walk the layers: behavioural tests, then the gate script comparing to the champion on quality, latency and a protected slice, then canary analysis for runtime behaviour. Trade-off: more layers mean slower releases and more false blocks. Measurable check: planted bad and slow candidates in CI, each blocked at the expected layer, plus the count of real blocks per quarter to show the gates are alive.

Q: Why is your gate fail-closed and what does that cost? **A:** A missing metric or an unreachable registry returns failure, so a broken pipeline cannot ship silently. The cost is blocked releases during outages and an on-call burden. Mitigation: a documented, audited manual override with expiry. Check: a drill with the registry disabled must end in a blocked release and an alert, not a pass.

20.4 Project 4: Monitoring, Drift and the Closed Retraining Loop

Conceptual Deep Dive

Goal. Close the loop: prediction logging, drift and quality jobs, metrics, alerts and a retrain trigger with debounce and a champion gate. This project demonstrates that you can distinguish noise from incidents, which is what monitoring interviews probe.

Architecture. A prediction log with request ID, features, score and model version; a drift job (a library or a custom PSI function, Module 08 section 3 (legacy)); Prometheus metrics with Grafana dashboards; Alertmanager sending a webhook into the training pipeline of 20.3; and a delayed-label join that computes quality when truth arrives (Module 08 section 3 (legacy)).

Milestones. M1: log predictions and labels. M2: compute drift and null-rate metrics per window. M3: inject covariate drift, a null flood and concept drift, then record which alerts fire and which do not. M4: tune thresholds from history instead of defaults. M5: connect an alert to a retrain trigger with cooldown and a champion comparison.

Acceptance tests. Alert precision and recall on injected incidents are above targets you state in advance; a seasonal pattern produces no page; one incident triggers exactly one retrain; a killed monitoring job raises a staleness alert.

Failure drills. The monitoring job dies, alerts flap around a threshold, and the retrain pipeline is fed bad data (the champion gate must reject the candidate).

Stretch. Estimate performance without labels (8.16) and compare it with delayed truth. Interview stories map to 8.17, 8.18, 5.16 and 11.18.

Edge cases: monitoring library APIs change between releases (check current docs, or keep a custom PSI as a dependency-free fallback) and simulated incidents are cleaner than real ones, so report what the simulation does not cover.

Cross-Connection Scenarios

- Burn-rate and error-budget alerting for the service itself: 8.17.
- Slice and calibration monitoring that belongs in the quality job: 8.18 and 17.5.
- The retrain trigger and debounce reuse the pipeline of 20.3 and Module 05 section 9 (legacy).
- Root-cause playbooks to use after an alert: 11.18.

Production Anti-Patterns

- Alerting on every drifted feature: with 200 features something always drifts, so on-call learns to ignore the page.
- No cooldown on retrain: one incident fires several jobs, each promoting a candidate trained on partially bad data.
- Monitoring job unmonitored: silence is read as health when the job simply stopped.

Minimal Working Code

An incident injector with ground truth for what should alert.

```
import numpy as np

def psi(ref, cur, bins=10):
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf
    r = np.histogram(ref, edges)[0] / len(ref) + 1e-6
    c = np.histogram(cur, edges)[0] / len(cur) + 1e-6
    return float(np.sum((c - r) * np.log(c / r)))

def make(seed=0, n=2000):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    return x, (x + rng.normal(scale=0.5, size=n) > 0) * 1

def inject(kind, seed=1, n=2000):
    x, y = make(seed, n)
    if kind == "drift":
        x = x + 1.5
    elif kind == "null_flood":
        x = np.where(np.random.default_rng(seed).random(n) < 0.4, np.nan, x)
    elif kind == "concept":
        y = 1 - y
    truth = {"drift": "drift", "null_flood": "null", "concep
t": "quality", "none": None}
    return x, y, truth[kind]

if __name__ == "__main__":
    ref, _ = make(0)
    for kind in ["none", "drift", "null_flood", "concept"]:
        x, y, expected = inject(kind)
        nulls = float(np.isnan(x).mean())
        d = psi(ref, x[~np.isnan(x)])
        print(f"{kind:10s} psi={d:.2f} null={nulls:.2f} should_alert={expected}")
    assert psi(ref, inject("drift")[0]) > 0.25 > psi(ref, inject("none")[0])
```

An alert evaluator that replays a series through debounce and cooldown.

```
def replay(series, thr=0.25, for_n=3, cooldown=10):
    fired, streak, quiet = [], 0, 0
    for t, v in enumerate(series):
        quiet = max(0, quiet - 1)
        streak = streak + 1 if v > thr else 0
        if streak >= for_n and quiet == 0:
            fired.append(t)
            quiet = cooldown
    return fired
```

```
def score(fired, incidents):
    tp = sum(any(a <= t <= b for t in fired) for a, b in incidents)
    fp = sum(not any(a <= t <= b for a, b in incidents) for t in fired)
    return tp / len(incidents), tp / max(1, tp + fp)

if __name__ == "__main__":
    series = [0.05] * 48
    series[12] = 0.4                    # one-window seasonal spike: must not page
    for t in range(20, 30):
        series[t] = 0.5                 # injected incident, hours 20-29
    fired = replay(series)
    recall, precision = score(fired, [(20, 29)])
    print(fired, recall, precision)
    assert fired == [22] and recall == 1.0 and precision == 1.0
```

Interview Angle

Q: How did you validate that your alerts are useful? **A:** I injected known incidents and replayed history through the rules, measuring recall on injected incidents and precision as pages that matched real or injected events. Trade-off: tighter thresholds raise precision and delay detection. Measurable check: recall at least a stated target, pages per week near zero on normal traffic, and a seasonal pattern that never pages. I would also review every real page for actionability monthly.

Q: Describe your retrain trigger and how it avoids storms. **A:** Trigger on a sustained signal (a breach lasting several windows), not a single spike; apply a cooldown and a lock so one incident yields one job; require the candidate to beat the champion on a holdout before promotion. Trade-off: slower reaction against fewer bad promotions. Check: one injected incident equals one retrain run, and a poisoned dataset is rejected by the champion gate.

20.5 Capstone: End-to-End Streaming Fraud Platform and Review Rubric

Conceptual Deep Dive

Goal. Integrate projects 1-4 into one platform for a single use case: streaming velocity features, online scoring with fallbacks, delayed labels, monitoring, gated retraining and canary rollout. The use case matters: a platform without a use case cannot be evaluated, and fraud forces every hard problem (low latency, delayed labels, adversaries, asymmetric costs).

Reference architecture. Transactions enter a stream; a stream job computes windowed velocity features (14.2) into the online store and an offline log; a scoring service reads features with tiered fallbacks (20.2); decisions and features are logged; labels arrive days later and are joined for quality; monitoring runs drift and quality jobs (20.4); the retrain pipeline trains, gates and registers (20.3); the canary rolls the new model out. The design is walked in 22.2. For a laptop, simulate the stream with a generator and a deterministic clock, and replace the cluster with processes.

Phased plan. Phase 1: batch features and online scoring. Phase 2: streaming features plus parity. Phase 3: monitoring and delayed labels. Phase 4: gated retraining, canary and failure drills. Prepare a 10-minute demo: problem and constraints (1 minute), architecture slide (2), live scoring path (2), one injected failure and its recovery (3), trade-offs and next steps (2).

Rubric (0-4 per axis). Reproducibility, data quality, parity, delivery safety, observability, cost awareness, documentation and failure drills. A 4 requires evidence you can show in the demo; a 2 means built but unverified; a 0 means absent. Weight delivery safety and failure drills most for platform roles.

Common review comments. "What happens if the stream lags?", "How do you know the features match?", "Why this threshold?", "What does it cost per thousand predictions?" Pre-empt them in the README with measured answers.

Presenting. One architecture slide, two defended trade-offs, one thing that broke and what changed afterwards, and what you would do at 50x traffic (see 12.16 for the arithmetic).

Edge cases: scope creep (freeze scope per phase and park ideas in a list) and demo fragility (record a fallback run; keep failure injection as a script, not a live typing exercise).

Cross-Connection Scenarios

- Streaming parity and backfills: 14.3; stateful scoring and dead letters: 14.5.

- Domain framing for fraud, including adversaries and costs: 18.2.
- Platform and cost framing for the presentation: 16.3 and 16.5.
- Practice the walkthrough with the scorecard in 21.4 and the design in 22.2.

Production Anti-Patterns

- Building the platform without a use case: components multiply, none is validated against real constraints.
- No failure injection in the demo: reviewers cannot distinguish a working system from a happy-path script.
- Over-claiming scale: simulated throughput presented as production capacity collapses under the first follow-up question.

Minimal Working Code

A rubric scorer with evidence requirements.

```
AXES = {"reproducibility": 2, "data_quality": 1, &
quot;parity": 2, "delivery_safety": 3,
        "observability": 2, "cost": 1, "documentation": 1, "failure_dr
ills": 3}

def score(scores, evidence):
    total, weight, notes = 0, 0, []
    for axis, w in AXES.items():
        s = scores.get(axis, 0)
        if s >= 3 and not evidence.get(axis, "").strip():
            s = 2
            notes.append(f"{axis}: capped at 2, no evidence")
        total += s * w
        weight += 4 * w
    weakest = sorted(AXES, key=lambda a: (scores.get(a, 0), -AXES[a]))[:3]
    return round(100 * total / weight, 1), weakest, notes

if __name__ == "__main__":
    s = {a: 3 for a in AXES}
    s["failure_drills"] = 1
    e = {a: "demo + test" for a in AXES}
    del e["cost"]
    pct, weakest, notes = score(s, e)
    print(pct, weakest, notes)
    assert weakest[0] == "failure_drills" and notes == ["cost: capped at 2, no evidence&qu
ot;]
```

A demo-script generator that time-boxes a 10-minute walkthrough.

```
def outline(parts, total_min=10):
    weight = sum(w for _, w in parts)
    t, lines = 0.0, []
    for name, w in parts:
        d = total_min * w / weight
        lines.append(f"{t:4.1f}-{t + d:4.1f} min  {name}")
        t += d
    return lines

PARTS = [("Problem and constraints", 1), ("Architecture slide", 2),
         ("Live scoring path", 2), ("Injected failure and recovery", 3),
         ("Trade-offs and next steps", 2)]

if __name__ == "__main__":
    for line in outline(PARTS):
        print(line)
    assert outline(PARTS)[-1].startswith(" 8.0-10.0")
```

Interview Angle

Q: Give a 5-minute architecture walkthrough of your capstone and defend two trade-offs. **A:** Open with the use case and constraints, then follow one transaction through features, scoring, logging and labels, then the operations loop. Defend two choices with alternatives: streaming versus micro-batch features (freshness against complexity) and fail-open versus fail-closed fallbacks (availability against fraud loss). Measurable check: parity mismatch rate, p99 against budget, and one injected failure's detection and recovery time.

Q: What would you change if traffic grew 50x? **A:** Re-run the capacity arithmetic first (12.16) to find which resource binds: stream partitions, online store memory or replicas. Likely changes: partition by key with hot-key handling, sharded or tiered storage, autoscaling on queue depth, and sampled logging. Trade-off: complexity and cost against headroom. Check: a load test at 1.5x the new target, with p99 and consumer lag inside their budgets.